

Before jumping into development, it's essential to understand the groundwork that supports your game. This part focuses on preparing your project, setting up your workspace, and getting all necessary tools and assets ready. Think of it as establishing a solid foundation—when this base is organized and stable, everything you build afterward becomes easier to manage. In this section, you'll set up the game environment, bring in assets such as models, textures, and audio, and create the initial scenes where your gameplay will unfold. You'll also structure your folders and GameObjects so the entire project remains clean and manageable as it expands.

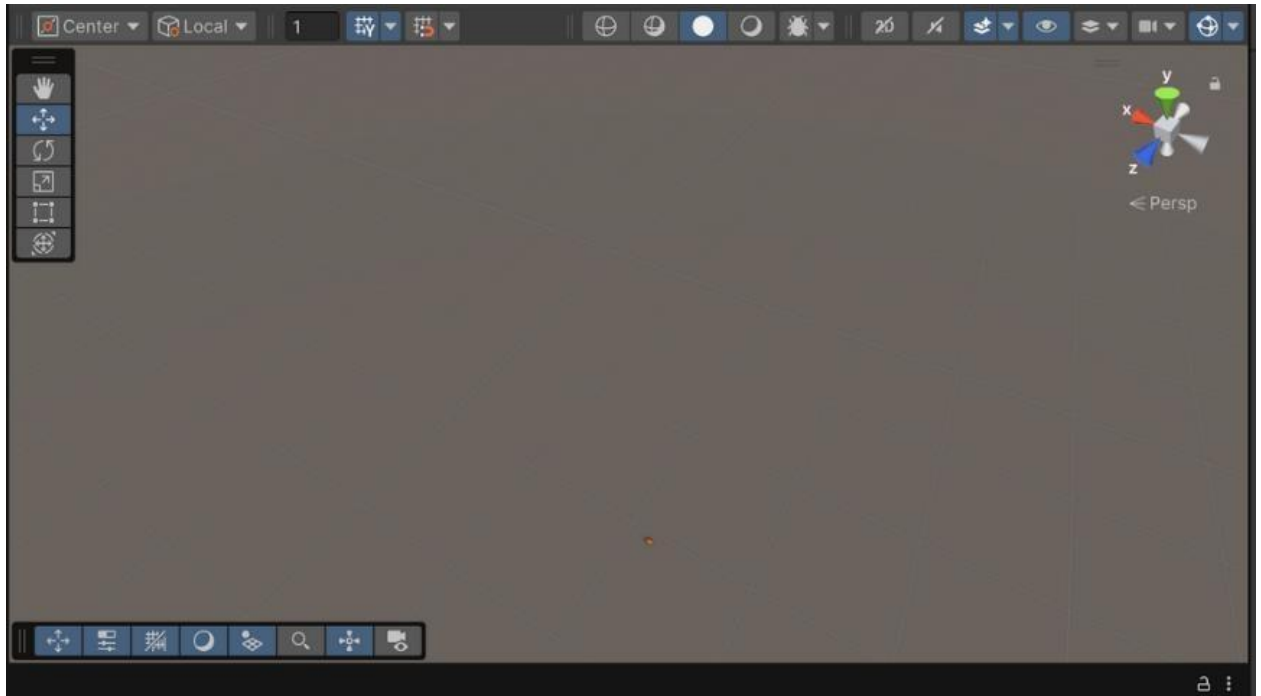
Once you finish this setup stage, you won't just have a fresh Unity project—you'll have a fully organized workspace and a clear direction for the development ahead, making the rest of the game-building process smoother and more enjoyable.

Part 1: Creating the Scene

Set up a new Unity project, add a base scene, and prepare all assets so that you can start building the game without errors.

Step 1: Create the Project

- 1 . Open Unity Hub on your computer.
 - Tip: Make sure you have the latest Unity version installed for best compatibility.
- 2 . Click New Project.
- 3 . In the project window:
 - Name the project Rockfall.
 - Choose 3D mode.
 - Note: 3D mode is required because we are working with 3D models and physics.
- 4 . Choose a location on your computer to save the project.
- 5 . Click Create Project.
- 6 . Wait for Unity to initialize the project.
 - Tip: Initialization may take a few minutes if it's the first time creating a project.



Step 2: Save the Scene

1. Go to the File menu.
2. Select Save As.
3. Navigate to the Assets folder.
4. Name the scene Main.scene.
5. Click Save.
6. Tip: Always save your scene at the start to avoid losing changes.

Step 3: Import the Assets

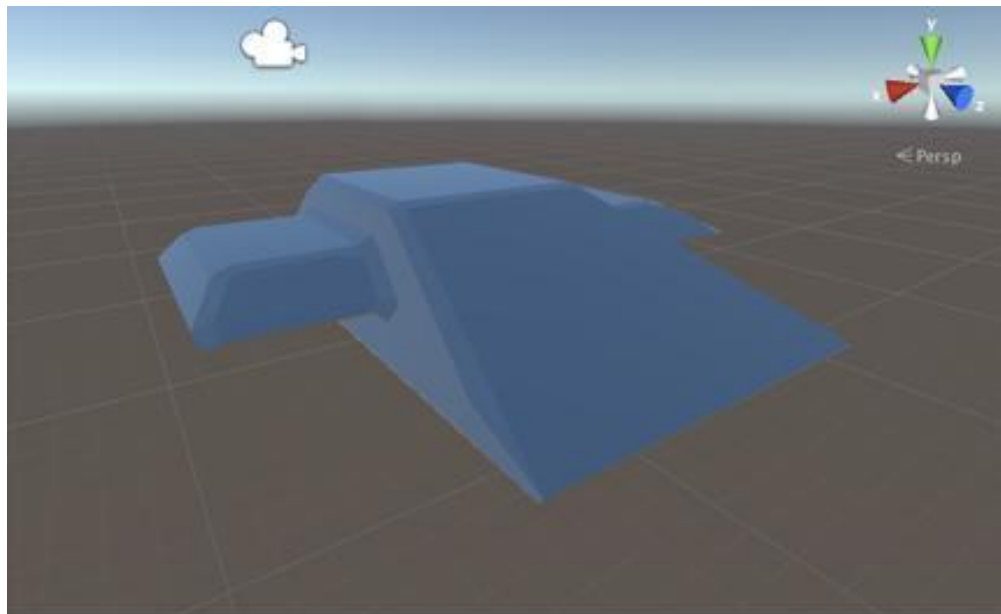
1. Locate the .unitypackage file you downloaded earlier (from “Getting the Assets” step).
2. Double-click the file. o Unity opens an Import Unity Package window.
3. Click Import All.
 - o This imports models, textures, materials, scripts, and prefabs.
4. Wait for all assets to finish loading.

5 . Verify the import:

- Open the Assets folder in Unity.
- Check that folders for Models, Scripts, Skybox, Prefabs exist.

Part 2: Adding the Ship

Add a spaceship that moves forward, has a collider, and is ready to attach scripts for player control.



Step 1: Create the Ship Object

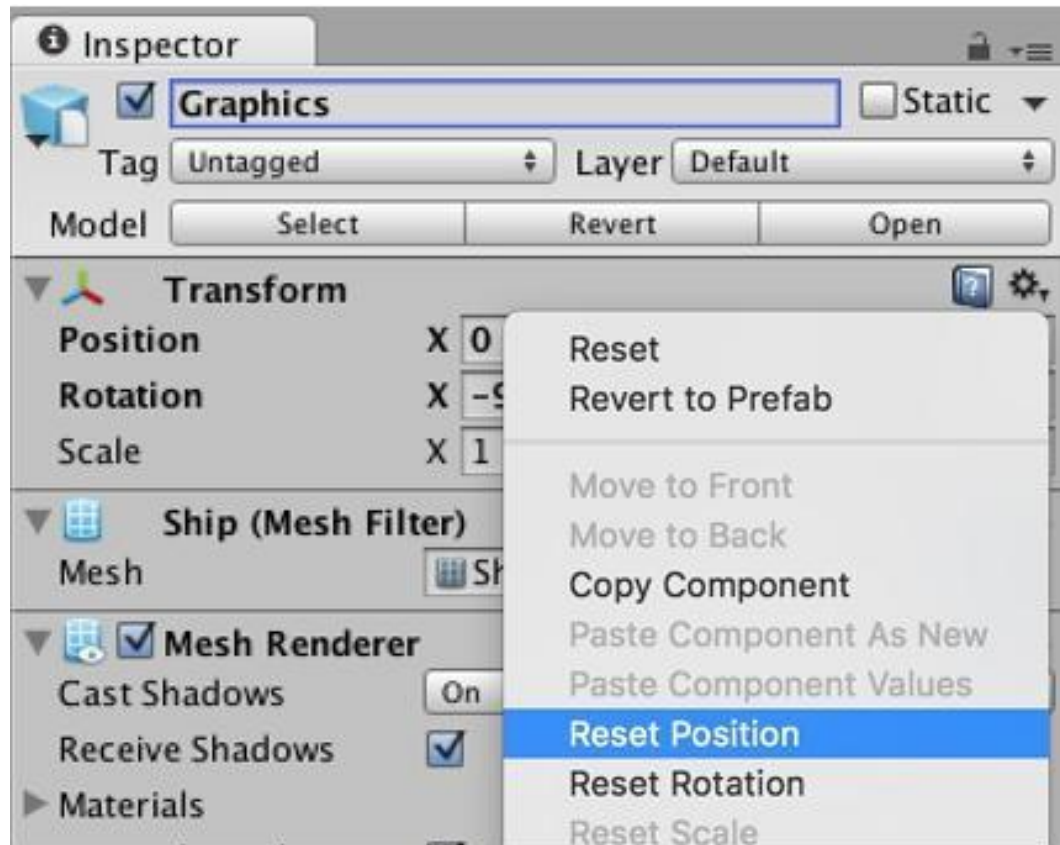
- 1 . Go to GameObject → Create Empty.
- 2 . A new empty GameObject appears in the hierarchy.
- 3 . Rename it Ship.
- 4 . Tip: Empty GameObjects are useful because they can hold multiple child objects, like models, colliders, and scripts.

Step 2: Add the Ship Model

- 1 . Open the Models folder.
- 2 . Drag the Ship model onto the Ship GameObject in the hierarchy.
 - o This automatically makes the model a child object of Ship.
- 3 . Rename the model object to Graphics.
- 4 . Tip: Keep the parent object empty with scripts, and child objects for visuals or effects, so it's easier to organize.

Step 3: Reset Position of the Graphics Object

- 1 . Select Graphics.
- 2 . Click the gear icon next to the Transform component.
- 3 . Choose Reset Position.
- 4 . Leave Rotation as (-90, 0, 0) to match Unity's coordinate system.
 - o Explanation: Blender's Z-up system differs from Unity's Y-up, so rotation is needed to align correctly.



Step 4: Add a Collider

- 1 . Select the Ship object (the parent object).
- 2 . Click Add Component → Physics → Box Collider.
- 3 . Enable Is Trigger.

o Note: Triggers detect collisions without stopping movement, useful for gameplay.

4.Set Size to (2, 1.2, 3).

5. Tip: The collider must match the ship's shape for proper collisions with asteroids or space stations.

Step 5: Add ShipThrust Script

- 1 . With Ship selected, click Add Component → New Script → C#.

- 2 . Name it ShipThrust.cs.
- 3 . Open the script in your editor (Visual Studio or VS Code).
- 4 . Add the following logic:

```
public float speed = 10f;

void Update() {

    transform.Translate(Vector3.forward * speed *
    Time.deltaTime);

}
```

- 5 . Save and return to Unity.
- 6.Tip: This moves the ship forward at a constant speed, independent of frame rate.

Step 6: Test the Ship Movement

- 1 . Click Play.
- 2 . Observe the ship moving forward automatically.
- 3 . Press Stop to exit Play Mode.
- 4 . Tip: If the ship isn't moving, check that the script is attached to **Ship**, not Graphics.

Part 3: Camera Follow

Make the camera follow the ship smoothly, like a real flight camera.

Step 1: Add SmoothFollow Script

- 1 . Select Main Camera.

2 . Click Add Component → New Script → C#.

3.Name it SmoothFollow.cs.

Step 2: Configure SmoothFollow

1 . Drag the Ship object into the Target field.

2 . Adjust optional settings:

- o Distance: how far the camera stays behind.

- o Height: how high the camera floats above the ship. o Smooth Speed: how fast the camera follows.

3 . Tip: Lower smooth speed = slower camera movement, more cinematic.

Step 3: Test the Camera

1 . Click Play.

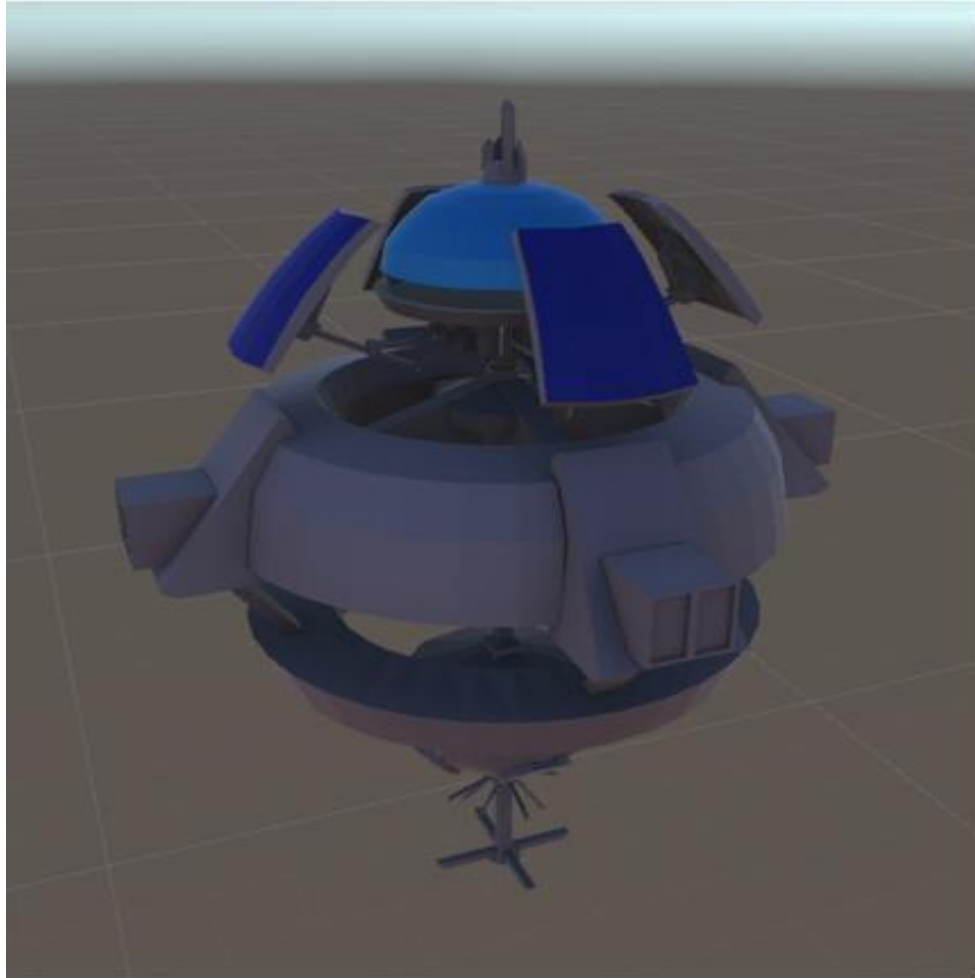
Observe:

- o The camera follows the ship smoothly. o Sharp turns create a slight lag, making it feel realistic.

3.Tip: Adjust distance and height until the ship is fully visible on screen.

Part 4: Adding the Space Station

Goal: Add a static space station with proper colliders to interact with asteroids.



Step 1: Create a Container

- 1 . Go to GameObject → Create Empty.
- 2 . Rename it Space Station.
- 3 . Tip: Using an empty object allows you to attach scripts or effects later.

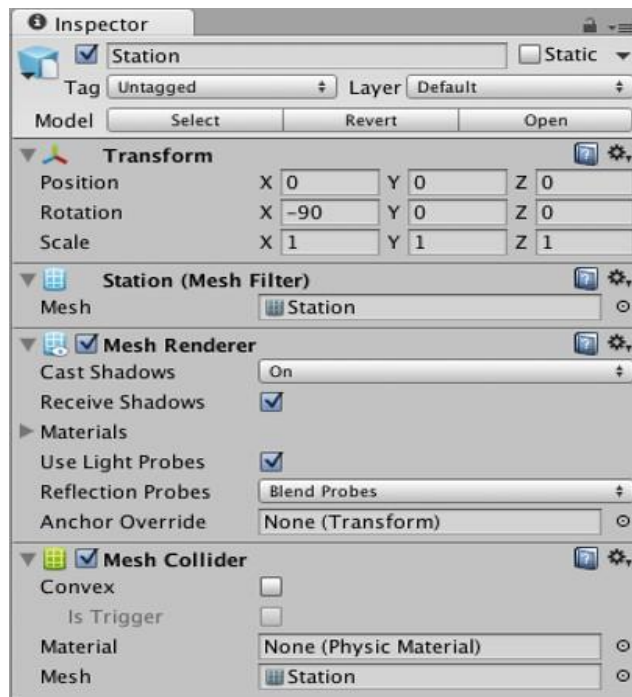
Step 2: Add the Model

- 1 . Open Models folder.
- 2 . Drag Station model onto Space Station.
- 3 . Reset its position using Transform → Reset Position.

- 4 . Optional: Adjust rotation or scale if needed.

Step 3: Check Colliders

- 1 . Expand the **Station** model in the hierarchy.
- 2 . Select the main mesh.
- 3 . Verify it has a Mesh Collider.
- 4 . If missing:
 - o Click Add Component → Physics → Mesh Collider.
- 5 . Tip: Mesh Colliders are important for accurate collisions with asteroids.

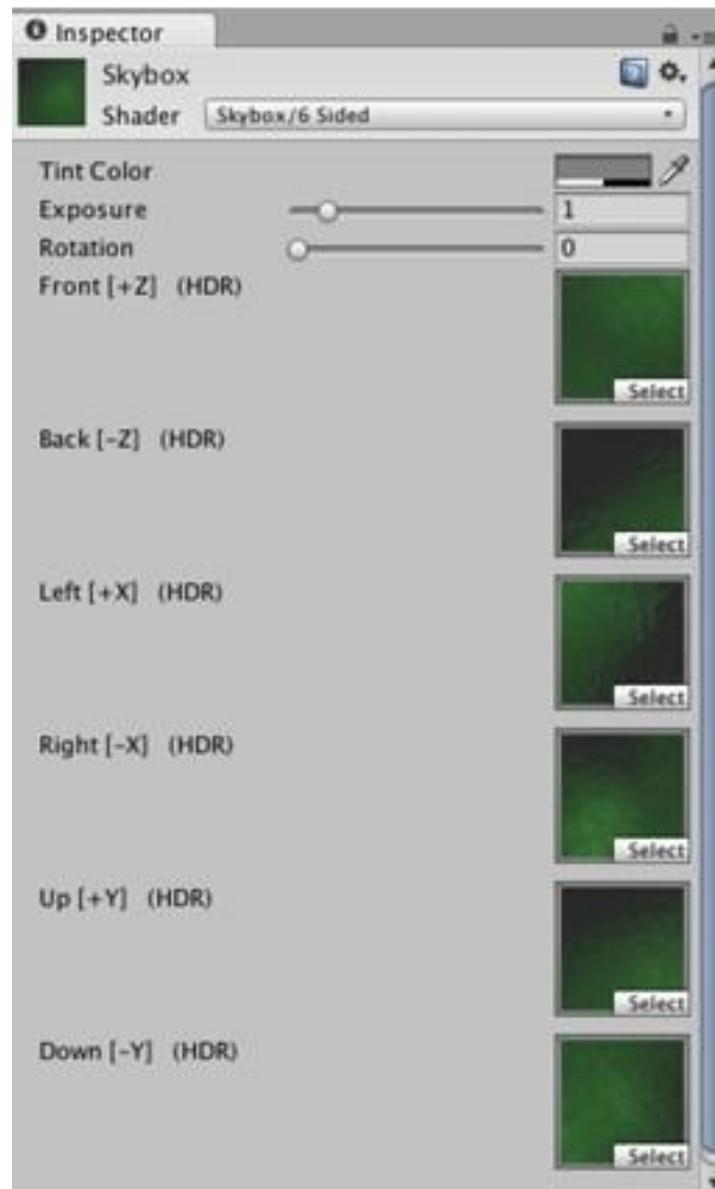


Part 5: Setting the Skybox

Replace the default Unity sky with a space skybox.

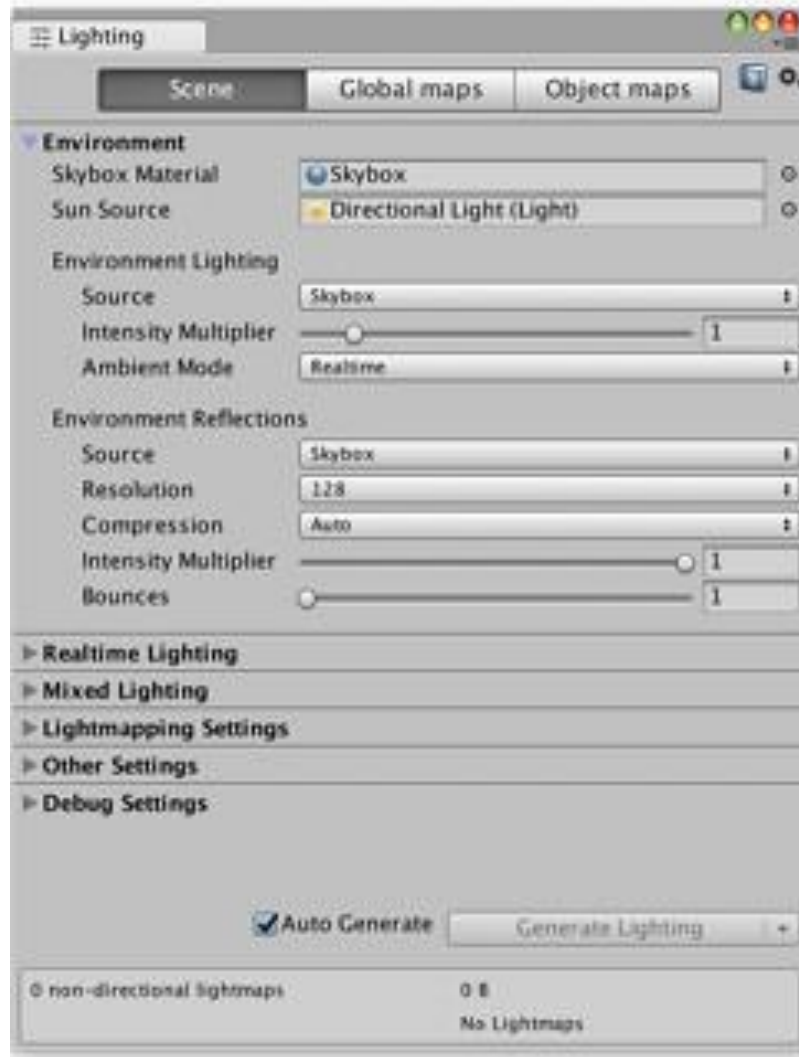
- Step 1: Create Skybox Material**
1. Open Assets → Create → Material.
 - 2 . Name it Skybox.

- 3 . Move it to the Skybox folder.



Step 2: Configure the Skybox Material

- 1 . Select the material.
- 2 . Change Shader → Skybox → 6 Sided.
- 3 . Drag six skybox textures into the slots: Up, Down, Front, Back, Left, Right.
- 4 . Tip: Use high-quality textures to avoid visible seams.



Step 3: Apply Skybox to Scene

- 1 . Open Window → Lighting → Settings.
- 2 . Drag the Skybox material into the Skybox slot.
- 3 . Observe:
 - The scene now has a realistic space background.
 - Lighting adjusts according to skybox colors.

Part 6: Adding the UI Canvas

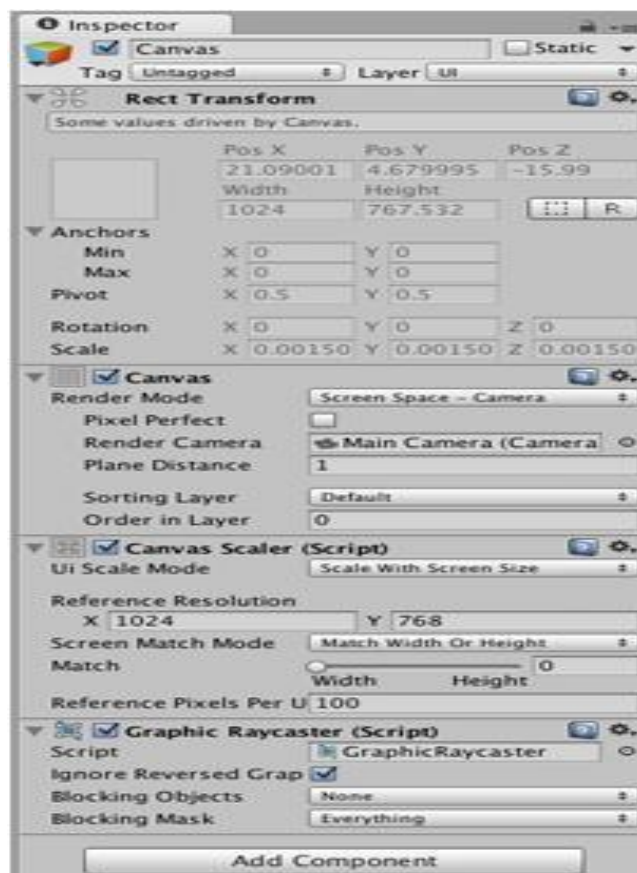
Prepare a canvas for flight controls, score, or health display.

Step 1: Create Canvas

1 . Go to GameObject → UI → Canvas.

2 . Unity automatically creates:

- Canvas
- EventSystem



Step 2: Configure Canvas

- 1 . Select Canvas.
- 2 . Set Render Mode → Screen Space - Camera.
- 3 . Drag Main Camera into Render Camera slot.
- 4 . Set Plane Distance → 1.
- 5 . Set UI Scale Mode → Scale with Screen Size.
- 6 . Set Reference Resolution → 1024 x 768.
- 7 . Tip: This ensures UI scales correctly on different devices.

Part 7: Wrapping Up

- Your scene now contains:
 - 1 . Ship moving forward
 - 2 . Smooth camera follow
 - 3 . Space station with colliders
 - 4 . Realistic skybox 5. Canvas ready for UI
- You can now add:
 - Player controls
 - Asteroids and obstacles
 - Shooting and explosions
 - Score and UI elements

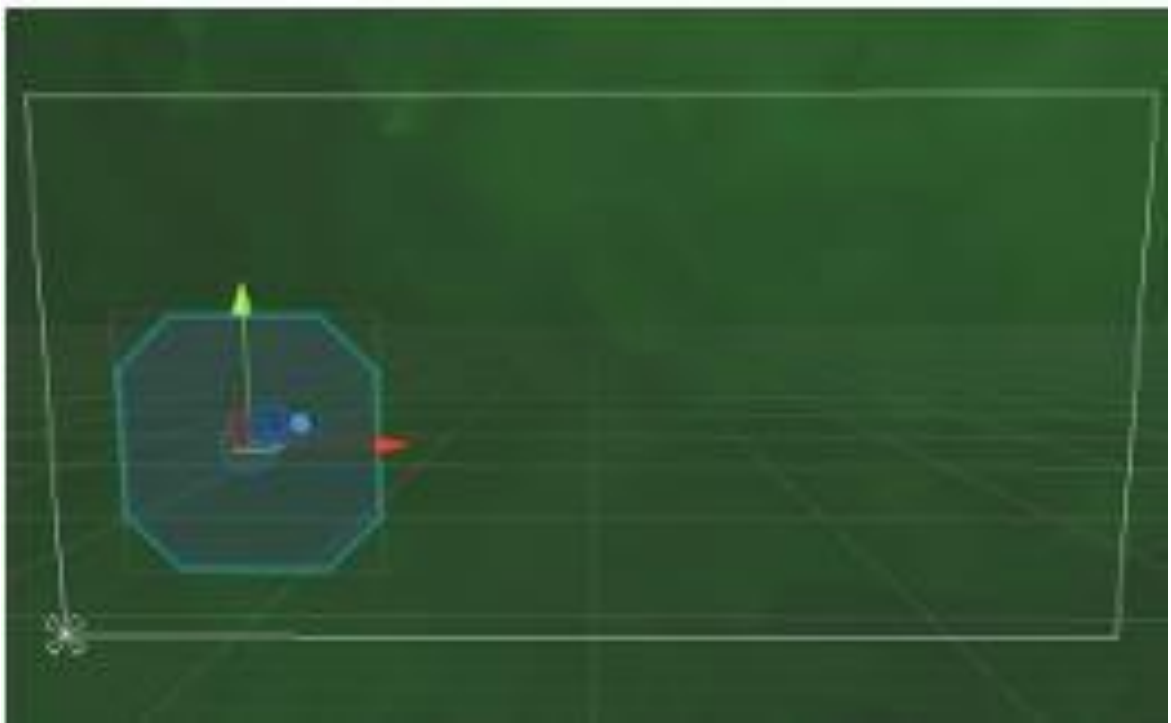
Input and Flight Control

The first step to creating a compelling space shooter is giving the player control over their ship. This section focuses on input handling and flight mechanics, teaching you how to translate user actions—like keyboard presses, mouse movements, or touchscreen gestures—into smooth, responsive movement within your game.

By the end of this section, your ship will respond intuitively to player input, letting them navigate the vastness of space with confidence. You'll have a solid foundation for all future gameplay elements, because every interaction in the game relies on tight, responsive control of the player's ship.

Part 1: Setting Up Input

Add a virtual joystick and input system to control the spaceship, supporting both directional flight and firing actions.



Step 1: Understanding Input

- 1 . The game will use two types of input:
 - Virtual joystick: provides directional input for flight.
 - Fire button: tells the ship when to shoot lasers.
- 2 . Tip: To properly test touchscreen input:
 - Use Unity Remote app on your mobile device. ○ If you test on a desktop without a touchscreen, clicks and drags will simulate the joystick, but behavior might slightly differ.

- 3 . Remember: The input system will eventually communicate with the ship through an Input Manager singleton, ensuring new ships instantiated from prefabs still receive input.

Part 2: Adding the Joystick

Create a visual joystick for the player to control ship movement.



Step 1: Create the Joystick Pad

- 1 . Go to GameObject → UI → Panel.
- 2 . Rename the panel Joystick.
- 3 . Make it square:
 - o Width: 250
 - o Height: 250
- 4 . Place it in the lower-left corner of the screen:
 - o Set anchors → Lower Left
 - o Position X = 125, Position Y = 125 (center of the square)
- 5 . Tip: Anchoring ensures the joystick stays in the corner regardless of screen resolution.

Step 2: Add the Pad Image

- 1 . Select the Joystick panel.
- 2 . In the Image component, set Source Image → Pad sprite.
- 3 . Verify visually that it looks like a square joystick pad.

Step 3: Create the Thumb

- 1 . Go to GameObject → UI → Panel again.
- 2 . Rename it Thumb.
- 3 . Make it a child of the Joystick panel.
- 4 . Adjust settings:
 - Anchors → Middle Center
 - Width = 80, Height = 80
 - Position X = 0, Y = 0
- 5 . Set Source Image → Thumb sprite.
- 6 . Tip: The thumb visually represents the joystick handle.

Step 4: Add the VirtualJoystick Script

- 1 . Select the Joystick object.
- 2 . Click Add Component → New Script → VirtualJoystick.cs.
- 3 . Script functionality:

- Implements `IBeginDragHandler`, `IDragHandler`,
`IEndDragHandler`.
 - On drag: moves the pad and thumb, stores delta value.
 - On release: resets pad, thumb, and delta.
- 4 . Tip: The delta property will be read by the Input Manager to steer the ship.

Step 5: Configure the Joystick Script

- 1 . Drag Thumb into the Thumb slot of the script.
- 2 . Tip: Ensure the thumb is properly referenced; otherwise, dragging won't visually move it.

Step 6: Test the Joystick

- 1 . Click Play.
- 2 . Drag inside the joystick pad:
 - Pad moves when drag starts.
 - Thumb follows your finger.
 - Delta value changes as you move the thumb.
- 3 . Adjust pad size or position if needed.

Part 3: Input Manager

Centralize input for all ships using a singleton so prefabs can receive joystick input.

Step 1: Create the Singleton Script

- 1 . In Assets, create a new `C#` script called `Singleton.cs`.
- 2 . Use the same template as Gnome's Well project:

- Ensures only one instance exists
 - Accessible from anywhere
- 3 . Tip: Singleton makes it easy for objects instantiated from prefabs to access the joystick.

Step 2: Create the Input Manager

- 1 . Go to GameObject → Create Empty. ○ Name it Input Manager.
- 2 . Add a new C# script: InputManager.cs.
- 3 . Open InputManager.cs and add:
 - Reference to VirtualJoystick
 - Optional future logic for firing lasers

Step 3: Configure Input Manager

- 1 . Drag the **Joystick** object into the **Steering** slot in **InputManager**.
- 2 . Tip: The ship will later read this reference every frame to determine flight direction.

Part 4: Flight Control

Make the ship respond to joystick input and move in a natural, intuitive way.

Step 1: Add ShipSteering Script

- 1 . Select Ship in the hierarchy.
- 2 . Add a new C# script called ShipSteering.cs.
- 3 . Script responsibilities:
 - Reads input from VirtualJoystick
 - Calculates smooth rotation

- Levels the ship gradually to prevent confusing rolls

Step 2: Test Steering

- 1 . Click Play.
- 2 . Drag inside the joystick area:
 - Ship turns according to input
 - Rolls back to a flat orientation when released
- 3 . Tip: Adjust rotation speed or roll correction for comfortable gameplay.

Part 5: Adding On-Screen Indicators

Show the positions of objects like space stations or asteroids with markers on the UI.

Step 1: Create Indicator Container

- 1 . Select Canvas.
- 2 . Go to GameObject → Create Empty Child.
- 3 . Name it Indicators.
- 4 . Set anchors → Stretch horizontally and vertically.

Step 2: Create a Prototype Indicator

- 1 . GameObject → UI → Image → rename Position Indicator.
- 2 . Make it a child of Indicators.

- 3 . Set Source Image → Indicator sprite.
- 4 . Create a Text label as a child of the indicator:

- Color = white
- Alignment = center
- Text = “50m”
- Font = CRYSTAL-Regular
- Font Size = 28

Step 3: Add Indicator Script

- 1 . Add a new C# script called Indicator.cs.
- 2 . Script logic:
 - Converts 3D world coordinates to viewport coordinates
 - Checks if the object is off-screen
 - Pushes markers to the edge if behind the camera
 - Converts to screen-space, then positions within the Indicators container
- 3 . Tip: This allows players to know where objects are, even off-screen.

Step 4: Connect the Distance Label

1. Drag the Text child object into the Distance Label slot of the script.

Step 5: Turn the Prototype into a Prefab

- 1 . Drag Position Indicator from hierarchy to Project pane.

- 2 . Delete the prototype from the scene.
- 3 . Tip: You can now instantiate multiple indicators at runtime for asteroids, stations, or other objects.

Part 6: Indicator Manager

Manage all indicators in one central object, ensuring prefabs can use them dynamically.

Step 1: Create Indicator Manager

- 1 . GameObject → Create Empty → rename Indicator Manager.
- 2 . Add a new C# script: IndicatorManager.cs.

Step 2: Configure Indicator Manager

- 1 . Drag **Indicators container** into **Label Container slot**.
- 2 . Drag Position Indicator prefab into Indicator Prefab slot.

Step 3: Attach Indicators to Objects

- 1 . Select Space Station.
- 2 . Add SpaceStation.cs script.
- 3 . Script function:
 - On start, request IndicatorManager to add an indicator tracking this object.
 - Tint indicator green for the station.
- 4 . Test by running the game: the station now has a visible indicator.

Wrapping Up

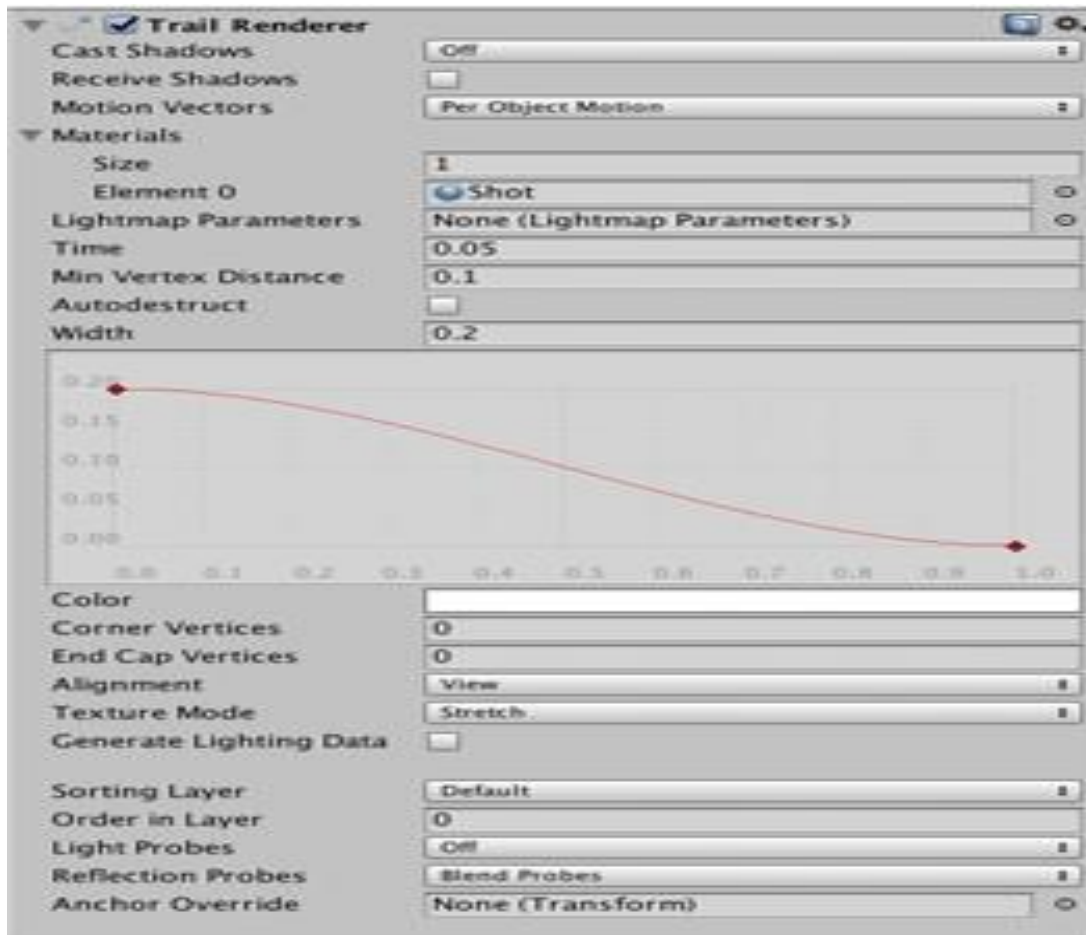
- Your game now has:
 - 1 . Virtual joystick input
 - 2 . Centralized Input Manager
 - 3 . Ship steering with smooth roll correction
 - 4 . On-screen indicators for objects
- Next steps include:
 - Adding asteroids
 - Implementing shooting and health systems
 - Creating dynamic targets with distance labels

Adding Weapons and Targeting

With flight and combat in place, the next step is to create dynamic challenges and consequences. This section focuses on asteroids, collisions, and the damage system, teaching you how to make the game world react to player actions and hazards.

You'll learn how to spawn asteroids, assign them movement patterns, and detect collisions with the ship or other objects. You'll also implement damage systems, where objects like the space station or the ship can take damage and eventually be destroyed. This adds stakes and tension: players must act strategically to survive and protect their objectives.

By the end of this section, the game will feel dangerous and exciting. Every asteroid is a threat, every collision matters, and the player must carefully balance offense and defense to progress. The world feels alive, reactive, and unpredictable, increasing the thrill of every play session.



Part 1: Creating Weapons and Targeting

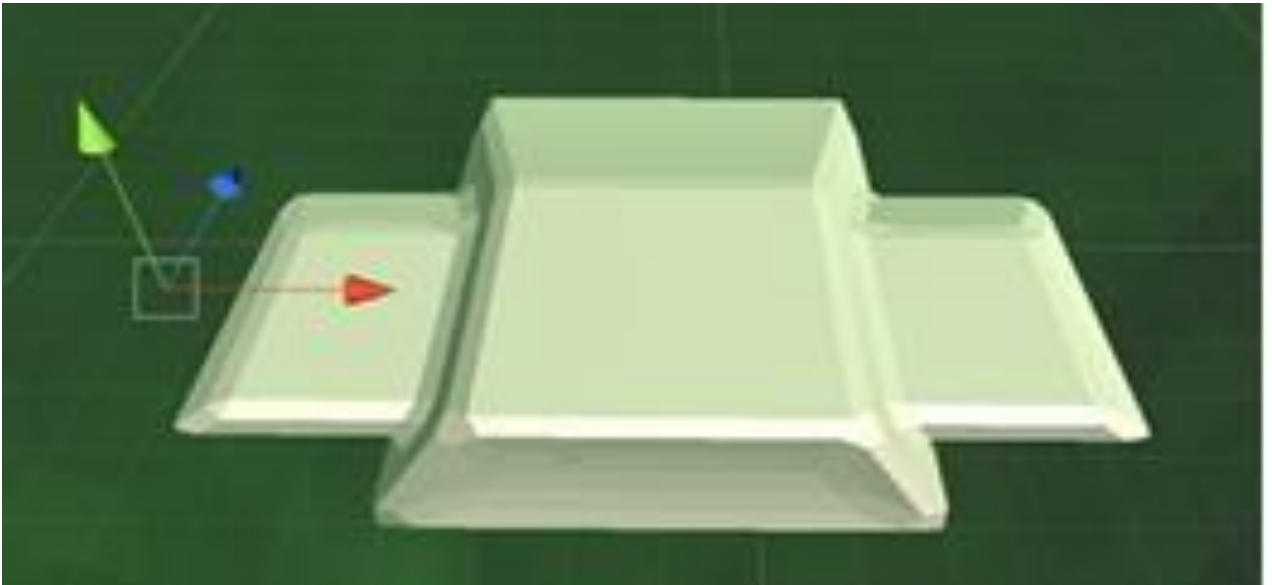
Make the spaceship capable of firing shots and provide a clear way for the player to aim.

Step 1: Creating the Shot

Objective: Create a projectile object that travels forward, deals damage, and disappears after a while.

- 1 . Create a new empty object in the scene.
 - o Name it Shot.
- 2 . Add Rigidbody Component:
 - o Turn off Use Gravity.
 - o Check Is Kinematic.
 - o Reason: We don't want shots affected by forces, but they still need collisions to be detected.
- 3 . Add Collider:
 - o Add a Sphere Collider.
 - o Radius = 0.5
 - o Center = (0,0,0)

- Enable Is Trigger.
- 4 . Add Shot Script:
- Create a new C# script called Shot.cs.
 - Responsibilities:
 - Move forward at a constant speed.
 - Destroy itself after life seconds.
 - Use Destroy(gameObject, life) in Start() to handle automatic removal.
 - Use transform.Translate(Vector3.forward * speed * Time.deltaTime) in Update() to move.



Step 2: Adding Visuals with Trail Renderer

Objective: Make the shot visually appealing using a trail effect.

- 1 . Create a Material:
 - Name it Shot.
 - Shader → Unlit/Color (no lighting).
 - Color → Bright red.
- 2 . Create Graphics Object:
 - Create an empty child object of Shot called Graphics.
 - Set position = (0,0,0).

3 . Add Trail Renderer:

- Component → Trail Renderer.
- Turn off Cast Shadows, Receive Shadows, Use Light Probes.
- Time = 0.05, Width = 0.2. ○ Curve: narrow toward the end. ○ Material → Shot material.

4 . Make Shot a Prefab:

- Drag Shot from hierarchy into Project → Objects folder.
- Delete it from the scene.

Step 3: Setting Up Ship Weapons

Objective: Make the spaceship capable of firing shots from multiple points.

1 . Add ShipWeapons Script:

- Select Ship. ○ Add new C# script ShipWeapons.cs.
- Responsibilities:
 - Holds shotPrefab reference.
 - Holds array of firePoints.
 - Tracks firePointIndex to alternate between left/right cannons.

Create Fire Points: Create empty object Fire Point 1 (child of Ship).

Position = (-1.9,0,0).

- Duplicate → Fire Point 2, position = (1.9,0,0).
- These mark where shots will spawn.

2 . Configure Script:

- Drag Shot prefab into Shot Prefab slot.
- Drag both Fire Points into Fire Points array (Inspector lock trick can help).

Step 4: Adding the Fire Button

Objective: Allow the player to fire shots by holding a button.

1 . Create Fire Button:

- GameObject → UI → Button → Name: Fire Button.
- Anchors & pivot → Bottom Right. ○ Position = (-50,50,0), Width = Height = 160. ○ Image → Button sprite, Type = Sliced.
- Text → “Fire”, Font = CRYSTAL-Regular, Size = 28, centered.
- Color → Hex = 3DFFD0FF (light cyan).

2 . Replace Button Behavior:

- Remove default Button component. ○ Add Event Trigger component.
- Add Pointer Down event → call InputManager.StartFiring().
- Add Pointer Up event → call InputManager.StopFiring().

3 . Connect Input Manager to ShipWeapons:

- ShipWeapons registers itself in InputManager when instantiated. ○ Deregisters when destroyed (OnDestroy with Application.isPlaying check). ○ InputManager handles the coroutine to fire shots repeatedly while the button is held.

4 . Test Fire Button:

Play the game → hold Fire button → shots should appear from the ship alternately.

Step 5: Adding a Target Reticle

Objective: Help the player aim by showing where shots will go.

1 . Create Target Object:

- Child of Ship → Name: Target.
- Position = (0,0,100) (in front of ship).

2 . Add ShipTarget Script:

o Responsibilities:

- Tells IndicatorManager to display a custom sprite.
- Keeps the indicator in front of the ship.

3 . Configure ShipTarget:

- o Drag Target Reticle sprite into Target Image slot.

4 . Test Reticle:

Play the game → move/rotate ship → crosshairs appear at aiming point.

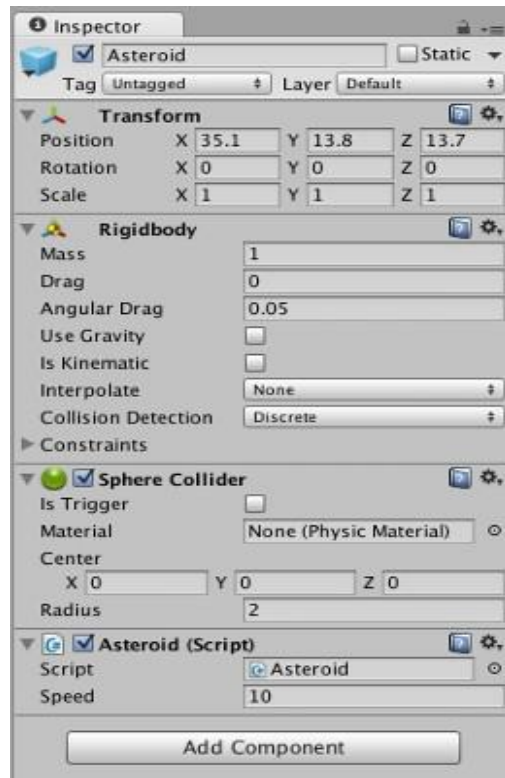
Part 2: Asteroids, Damage, and Explosions

So far, you can fly your ship, aim, and shoot laser bolts. But there's still nothing to shoot at—the space station doesn't count! To make the game more interesting, we'll create asteroids as targets, add a damage system so objects can take and deal damage, and finish with explosions to make the destruction satisfying.

Step 1: Creating Asteroids

Asteroids are the main targets in our game. They will move through space and can collide with the space station or be destroyed by the ship's lasers. Each asteroid will have two parts:

- 1 . The main asteroid object – contains all the logic, collision detection, and movement.
- 2 . The graphics object – the visible part of the asteroid, so the player can see it.



Creating the Asteroid Object

- 1 . In Unity, create a new empty GameObject and name it Asteroid.
- 2 . Find the asteroid model in your Models folder, and drag it onto the Asteroid object.
- 3 . Rename this child object Graphics.
- 4 . Reset the position of the Graphics object to (0,0,0) so it's perfectly centered on the parent.

Why separate graphics from logic?

Keeping the visual part separate allows you to adjust the look of the asteroid (like rotation, size, or texture) without affecting the game mechanics, collision detection, or movement.

Adding Physics

- 1 . Add a Rigidbody component to the parent Asteroid object. This allows Unity to detect collisions.
- 2 . Turn Gravity off, because space has no gravity.
- 3 . Add a Sphere Collider to the parent, and set the radius to 2. This determines the size of the area that detects collisions.

The Graphics child does not need physics components, because it's only for visuals.

Step 2: Asteroid Spawner

To make the game dynamic, we need more than one asteroid. Instead of manually placing each asteroid, we'll use an asteroid spawner to automatically create asteroids in random positions around the space station and send them flying toward it.

Creating the Spawner

- 1 . Create a new empty GameObject in the scene, and name it AsteroidSpawner.
- 2 . Set its position to where you want the asteroids to appear around the space station.
- 3 . Flatten the spawner by setting its scale to (1, 0.1, 1). This ensures asteroids appear mostly in a circle around the station rather than in a 3D sphere, making it easier to predict their paths.

Configuring the Spawner

- Assign the Asteroid prefab you just created to the spawner.
- Assign the Space Station as the target.
- Turn on Spawn Asteroids.

When you play the game now, asteroids will appear automatically and move toward the space station. You can adjust the spawn rate and speed to make the game easier or harder.

Step 3: Damage System

Right now, the ship can shoot, and the asteroids move, but nothing actually happens when they collide. To make the game interactive, we'll implement a damage system, allowing objects to take damage and be destroyed.

How Damage Works

- Each object that can take damage has a hit points value, which represents its health.
- When an object is hit, its hit points decrease.
- When hit points reach zero, the object is destroyed.
- Some objects can also deal damage, like shots or asteroids colliding with the space station.

Adding Damage to Asteroids

- 1 . Select the Asteroid prefab.

- 2 . Add a DamageTaking component to it. This allows the asteroid to receive damage from the ship's shots.
- 3 . Set the asteroid's hit points to 1. This makes it easy to destroy.

Adding Damage to Shots

- 1 . Select the Shot prefab.
- 2 . Add a DamageOnCollide component, so it can deal damage to anything it hits.

Adding Damage to the Space Station

- 1 . Select the Space Station object.
- 2 . Add a DamageTaking component. This allows it to take damage if an asteroid hits it.
- 3 . Enable the option for Game Over On Destruction. This doesn't do anything yet, but later it will end the game when the station is destroyed.

Test the game now: fly around and shoot asteroids. They should disappear when hit, and the space station can take damage if an asteroid collides with it.

Step 4: Adding Explosions

It's not satisfying for asteroids to just vanish. Let's make them explode when destroyed using particle effects.

Step 4.1: Create a Dust Material

- 1 . Open the Assets menu → Create → Material, and name it Dust.
- 2 . Set its Shader to Particles/Additive.
- 3 . Drag a dust texture into the Particle Texture slot.
- 4 . Set the Tint Color to a semi-transparent dark gray.

This material will be used for the lingering dust effect after the explosion.

Step 4.2: Create the Explosion Object

- 1 . Create a new empty object, name it Explosion.

- 2 . Create a child object named Fireball. This will create the initial flash of the explosion.
- 3 . Add a Particle System to the Fireball:
 - o The particles should appear bright, expand quickly, and fade out rapidly. o Adjust the color and size so it looks like a short, bright flash.
- 4 . Create another child object of Explosion named Dust: o Add a Particle System to the Dust object. o Assign the Dust material created earlier. o Configure the particles to be slower, darker, and fade gradually, simulating lingering smoke or dust.
- 5 . Convert the Explosion object into a Prefab, then remove it from the scene.
- 6 . Assign the Explosion prefab to the Asteroid prefab as its Destruction Prefab.

Test the game: now when you shoot asteroids, they explode with a bright flash and a cloud of dust.

Wrapping Up

By now, your game has:

- Asteroids that appear automatically and move toward the space station
- A working damage system: shots and asteroids can deal and take damage
- Explosions when asteroids are destroyed, making the game visually satisfying

Your space shooter is starting to feel like a complete game, with moving targets, hazards, and visible feedback for the player's actions.

Next steps could include: adding scoring, sound effects, multiple asteroid sizes, or different asteroid behaviors.

Audio, Menus, Death, and Explosions

Finally, no game feels complete without polish, feedback, and a sense of completion. This section is all about adding audio effects, visual flair, menus, and end-of-game logic, creating a satisfying and immersive experience.

You'll explore how to make engines roar, lasers zap, and explosions boom, bringing your game to life through sound. Menus and UI allow players to start a new game, pause, or continue after failure, making your game navigable and professional. You'll also handle game over conditions, like destroying the space station or leaving the play area, giving the game clear consequences and rewards.

By the end of this section, your game won't just work but it will also look, sound, and feel complete, delivering a polished, thrilling experience. Players will have the full range of interactions, from piloting and fighting to navigating menus and reacting to in-game events, making the game feel like a true space adventure

Part 1: Setting Up Menus

Why Menus Are Important

Right now, gameplay begins immediately when you press Play in the Unity editor. If the space station is destroyed, you have to stop and restart the game. Menus give players context, allow them to start a new game, pause, or restart after a loss, and make your game feel polished.

Types of Menus

Your game will have four main UI groups, and only one is visible at a time:

- 1 . Main Menu – Shows the game title and a “New Game” button.
- 2 . Paused Screen – Shows the text “Paused” and a button to unpause.
- 3 . Game Over Screen – Shows “Game Over” and a “New Game” button.
- 4 . In-Game UI – Displays the joystick, indicators, Fire button, and everything visible while playing.



Step 1: Organizing the In-Game UI

- 1 . Select your Canvas object in the Hierarchy.
- 2 . Create a new empty child object and name it In-Game UI.

- 3 . Set its anchors to stretch both horizontally and vertically, with all margins set to 0, so it fills the canvas.
- 4 . Move all existing UI elements (joystick, fire buttons, indicators, etc.) into the In-Game UI container.
- 5 . Disable the In-Game UI for now, so you can focus on building the other menus.

Step 2: Creating the Main Menu

- 1 . Create a new empty object called Main Menu under the Canvas. Stretch it to fill the canvas.
- 2 . Add a Text object as a child called Title. Set the anchor to Center Top, position Y = -100, width 1024, height 120. Set the text to your game's title, e.g., "Rockfall". Set the color to a light yellow (#FFE99A) and enable Best Fit so the text adjusts automatically.
- 3 . Add a Button object called New Game. Anchor it Center Top, X = 0, Y = -300, height = 80. Set its image and font. Set the button text to "New Game". 4. Disable the Main Menu container for now.



Step 3: Creating the Pause Menu

- 1 . Create an empty object called Paused. Stretch it to fill the canvas.
- 2 . Add a Text object called Title, with the text "Paused".
- 3 . Add a Button object called Unpause Button with the text "Unpause".
- 4 . Disable the Paused container.



Step 4: Creating the Game Over Menu

- 1 . Create an empty object called Game Over. Stretch it to fill the canvas.
- 2 . Add a Text object called Title, with the text "Game Over".
- 3 . Add a Button object called New Game Button with the text "New Game".
- 4 . Disable the Game Over container.

These menus may seem repetitive now, but separating them allows you to customize them individually later without breaking other menus.



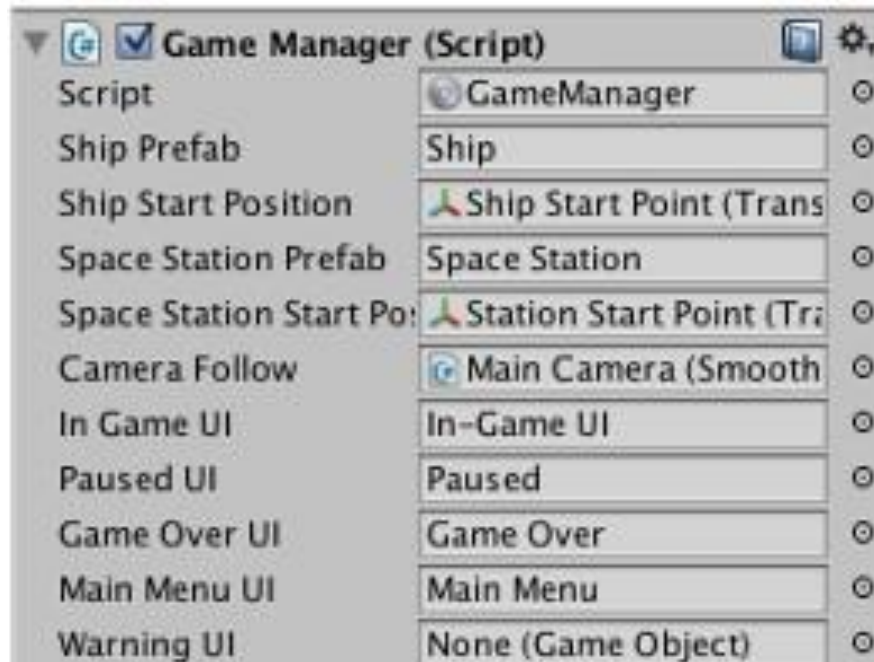
Step 5: Adding a Pause Button

- 1 . Under In-Game UI, create a new Button called Pause Button.
- 2 . Anchor it Top Right, X = -50, Y = -30, width = 80, height = 70.
- 3 . Set the text to "Pause". Use a bright color (#3DFFD0FF) and a readable font.

Now your UI is visually ready, but it won't function until you set up the Game Manager.

Step 6: Game Manager and Game Flow

The Game Manager controls everything about the game's state: menus, starting and ending games, pausing, and spawning ships and stations.



Step 7: Preparing Ship and Space Station

- 1 . Turn both the Ship and Space Station into prefabs by dragging them into the Project pane.
- 2 . Remove them from the scene.
- 3 . Create start position markers for each:
 - Ship Start Point – where the ship will spawn.
 - Station Start Point – where the station will spawn.
These are invisible in-game but visible in the editor for positioning.

Step 8: Setting Up the Game Manager

- 1 . Create an empty object called Game Manager.
- 2 . Connect references in the Inspector:
 - Ship and Station prefabs
 - Ship and Station start positions

- Main Camera (for camera follow)
- In-Game UI, Main Menu UI, Paused UI, Game Over UI
- Asteroid Spawner
- Warning UI (for boundaries)

Step 9: Connecting Buttons to Game Manager

- 1 . Pause Button → Call SetPaused(true)
- 2 . Unpause Button → Call SetPaused(false)
- 3 . New Game Buttons (Main Menu and Game Over) → Call
StartGame()

Now the Game Manager can handle starting, pausing, and ending the game, and switch menus accordingly.

Step 10: Game Over Logic

- 1 . The Space Station has a DamageTaking script. Enable Game Over On Destroyed to trigger the Game Over menu when it's destroyed.
- 2 . Add DamageOnCollide to asteroids so they can damage the station.
- 3 . Modify asteroid indicators to show the distance to the station so players know which to prioritize.

Step 11: Boundaries and Warnings

To prevent players from flying too far away:

- 1 . Create a Boundary object.
- 2 . Set up two spheres:
 - Warning Sphere – shows a red border when the ship goes too far.

- Destroy Sphere – ends the game if the ship leaves this area.
- 3 . Add a Warning UI image stretched to the canvas, using a sliced warning sprite.
- 4 . Configure the Game Manager to check the ship's distance every frame. If it crosses the warning radius, show the warning. If it crosses the destroy radius, trigger Game Over.

Part 2: Final Polish – Visual Effects

Step 1: Space Dust Add a Dust particle system as a child of the Ship prefab.

- 1 . Set its Simulation Space to World and shape to Sphere.
- 2 . This creates floating dust particles that the ship flies past, giving a sense of speed.

Step 2: Trail Renderers

- 1 . Create a Trail material using an Additive shader.
- 2 . Add Trail Renderer components to the ship for each engine.
- 3 . Duplicate for the second engine and apply to prefab.
- 4 . Add a Trail Renderer to asteroids' graphics for color trails.

These effects add visual flair and make movement easier to perceive.

Step 3: Ship Engine

- 1 . Add an Audio Source to the Ship prefab.
- 2 . Enable **Loop** and drag in the engine sound.

Step 4: Weapon Effects

- 1 . Add Audio Sources to the ship's fire points.
- 2 . Assign the laser sound effect.
- 3 . Make the script play the sound whenever the weapon fires.

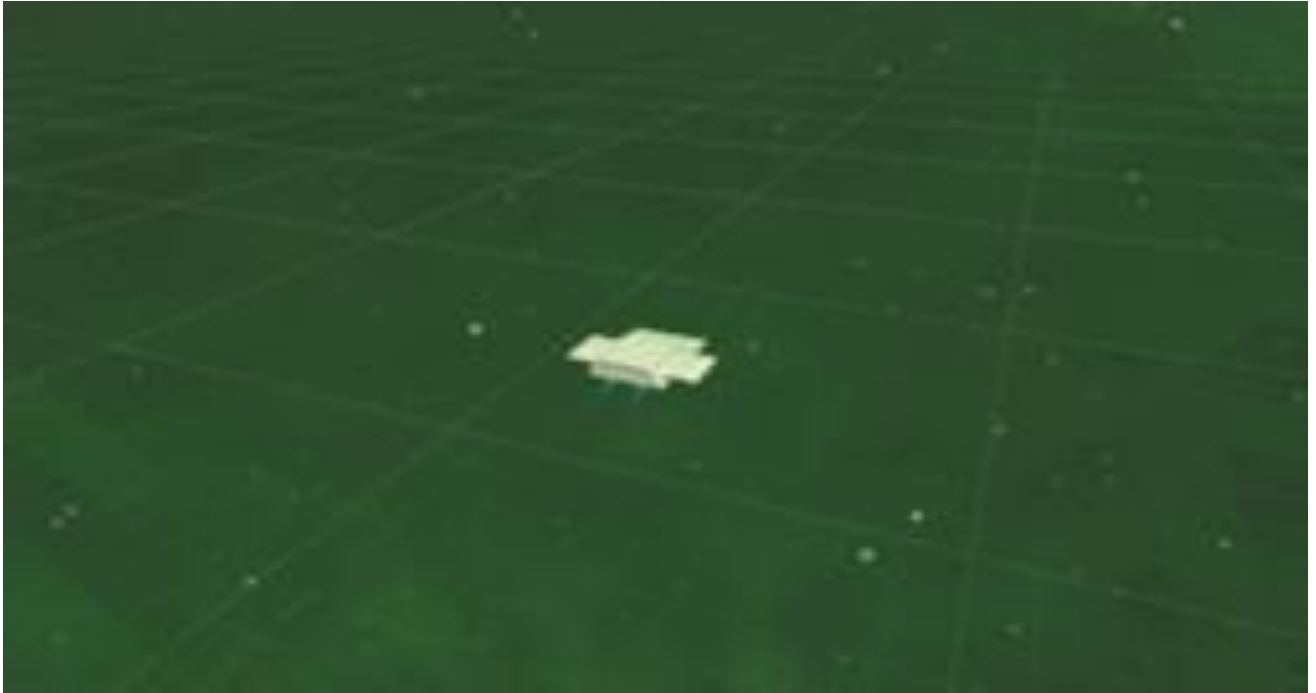
Step 5: Explosions

- 1 . Add an Audio Source to the Explosion prefab.
- 2 . Enable Play On Awake and assign the explosion sound.
- 3 . Now all explosions produce sound automatically.

Wrapping Up

Congratulations! Your game now has:

- Fully functional menus (Main, Pause, Game Over, In-Game)
- Game management with spawning, starting, pausing, and ending
- Boundary warnings to prevent players from flying too far
- Polished visuals: engine trails, asteroid trails, dust particles
- Audio: engine loops, laser sounds, explosions You're ready to play Rockfall as a complete game!



Conclusion

Rockfall demonstrates how to build a full 3D space shooter from start to finish using Unity. It combines movement, combat, UI, audio, and visual effects to create a polished, playable game.

By following this guide, you've learned not only the technical skills required but also the design thinking and problem-solving processes that go into making a complete game. Rockfall is now a foundation on which you can build, expand, and create new interactive experiences.